VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Maanya Chawla (1BF24CS158)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by Maanya Chawla **(1BF24CS158)**, who is a bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Namratha M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1: Write a program to obtain the Topological ordering of vertices in a given digraph.

C program :

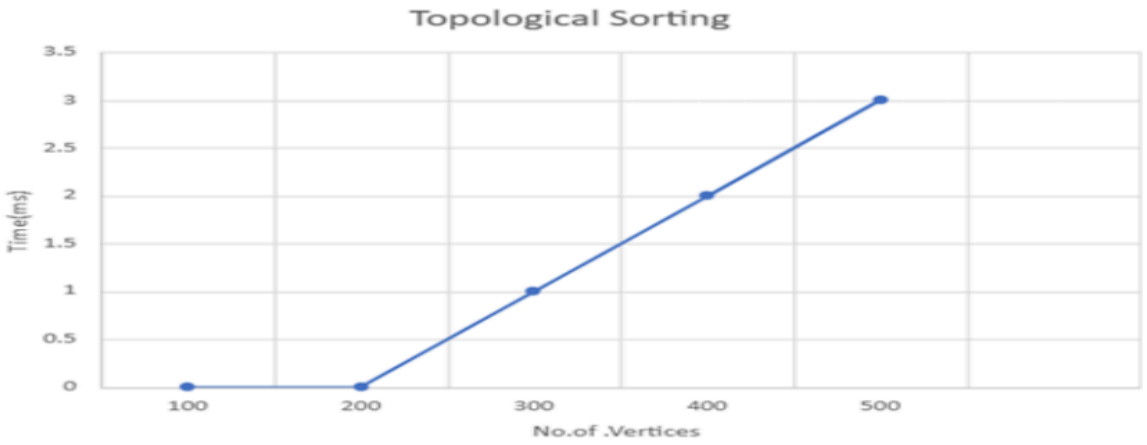
```
#include <stdio.h>
#define MAX 100
int main() {
    int n, e;
    int graph[MAX][MAX] = {0};
    int indegree[MAX] = {0};
    int queue[MAX], front = 0, rear = 0;
    int topo[MAX], count = 0;

    printf("Enter no. of vertices: ");
    scanf("%d", &n);

    printf("Enter no. of edges: ");
    scanf("%d", &e);

    printf("Enter edges (u, v)\n");

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);

        graph[u][v] = 1;
        indegree[v]++;
    }

    for (int i = 1; i <= n; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }

    while (front < rear) {
        int u = queue[front++];
        topo[count++] = u;

        for (int v = 1; v <= n; v++) {
            if (graph[u][v]) {
                indegree[v]--;

                if (indegree[v] == 0) {
                    queue[rear++] = v;
                }
            }
        }
    }

    if (count != n) {
        printf("Graph has a cycle. Topological sort not possible.\n");
    } else {
        printf("Topological Order: ");
```

```
        for (int i = 0; i < count; i++) {
            printf("%d ", topo[i]);
        }

        printf("\n");
    }
    return 0;
}}
```

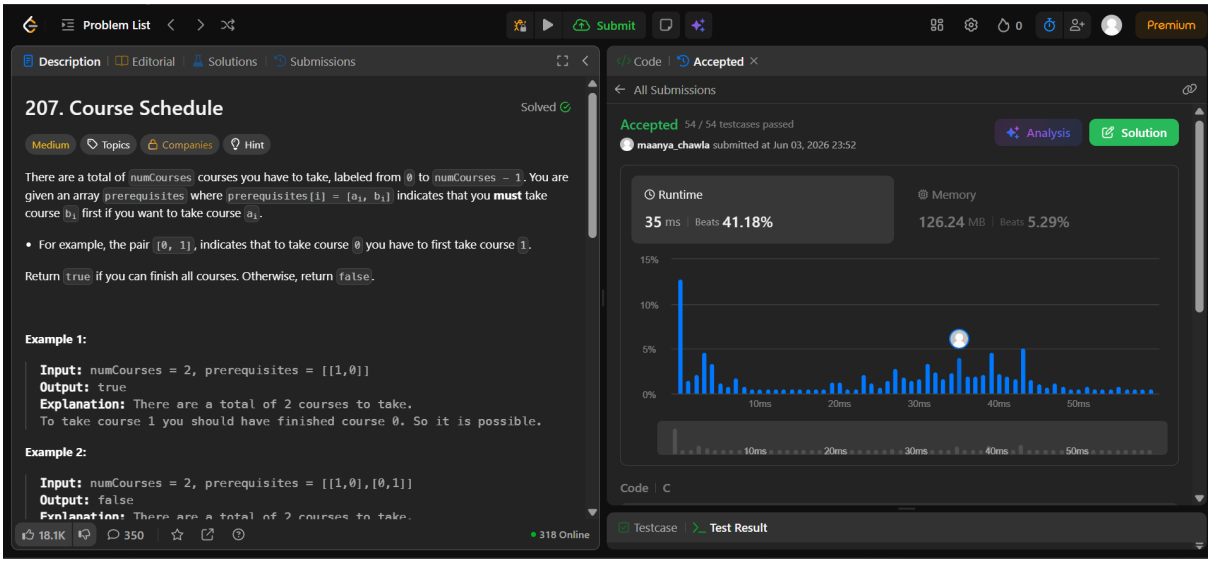
Output :

```
Enter no. of vertices: 5
Enter no. of edges: 5
Enter edges (u, v)
1 2
1 3
2 5
3 4
4 5
Topological Order: 1 2 3 4 5
```

Graph :



Leetcode 207 :



Code :

```
bool canFinish(int numCourses, int** prerequisites,
               int prerequisitesSize, int* prerequisitesColSize) {

    int *indegree = (int *)calloc(numCourses, sizeof(int));
    int **graph = (int **)malloc(numCourses * sizeof(int *));
    int *graphSize = (int *)calloc(numCourses, sizeof(int));

    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int *)malloc(prerequisitesSize * sizeof(int));
    }

    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        int prereq = prerequisites[i][1];

        graph[prereq][graphSize[prereq]++] = course;
        indegree[course]++;
    }

    int *queue = (int *)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;

    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }

    int count = 0;
```

```
    while (front < rear) {
        int curr = queue[front++];
        count++;

        for (int i = 0; i < graphSize[curr]; i++) {
            int next = graph[curr][i];

            indegree[next]--;

            if (indegree[next] == 0) {
                queue[rear++] = next;
            }
        }
    }

    for (int i = 0; i < numCourses; i++) {
        free(graph[i]);
    }

    free(graph);
    free(graphSize);
    free(indegree);
    free(queue);

    return count == numCourses;
}
```

Lab program 2: Implement Johnson Trotter algorithm to generate permutations.

C Program :

```
#include <stdio.h>

#define MAX 10
#define L -1
#define R 1

void print(int a[], int n) {
   for (int i = 0; i < n; i++)
      printf("%d ", a[i]);
   printf("\n");
}

int main() {
   int n, a[MAX], dir[MAX];

   printf("Enter n: ");
   scanf("%d", &n);

   for (int i = 0; i < n; i++) {
      a[i] = i + 1;
      dir[i] = L;
   }

   print(a, n);

   while (1) {
      int mobile = 0, pos = -1;

      // find largest mobile
      for (int i = 0; i < n; i++) {
         if (dir[i] == L && i > 0 &&
            a[i] > a[i-1] && a[i] > mobile) {
            mobile = a[i];
            pos = i;
         }

         if (dir[i] == R && i < n-1 &&
            a[i] > a[i+1] && a[i] > mobile) {
            mobile = a[i];
            pos = i;
         }
      }

      if (mobile == 0)
         break;

      int next = pos + dir[pos];

      // swap
      int t = a[pos];
      a[pos] = a[next];
```

```
        a[next] = t;

        t = dir[pos];
        dir[pos] = dir[next];
        dir[next] = t;

        // reverse larger elements
        for (int i = 0; i < n; i++)
            if (a[i] > mobile)
                dir[i] *= -1;

        print(a, n);
    }

    return 0;
}
```

Output :

```
Enter n: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3
```

Lab program 3: Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

C Program :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int a[], int low, int mid, int high)
{
   int i = low;
   int j = mid + 1;
   int k = 0;
   int b[100000];

   while(i <= mid && j <= high)
   {
      if(a[i] < a[j])
         b[k++] = a[i++];
      else
         b[k++] = a[j++];
   }

   while(i <= mid)
      b[k++] = a[i++];

   while(j <= high)
      b[k++] = a[j++];

   for(i = low, k = 0; i <= high; i++, k++)
      a[i] = b[k];
}
void mergesort(int a[], int low, int high)
{
   if(low < high)
   {
      int mid = (low + high) / 2;

      mergesort(a, low, mid);
      mergesort(a, mid + 1, high);

      merge(a, low, mid, high);
   }
}

int main()
{
   int a[100000], n;
   int values[5] = {100, 500, 1000, 2000, 3000};

   clock_t start, end;
   double time_taken;
```

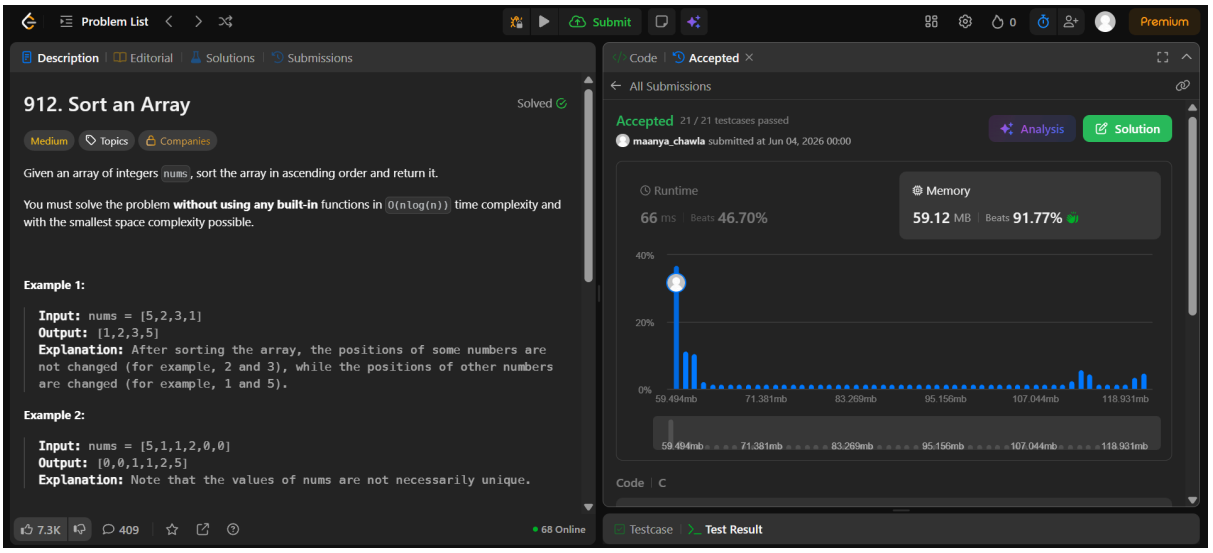
```
    srand(time(NULL));

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    mergesort(a, 0, n - 1);

    printf("\nSorted elements:\n");

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n\nN\tTime(ms)\n");

    for(int i = 0; i < 5; i++)
    {
        n = values[i];

        for(int j = 0; j < n; j++)
            a[j] = rand() % 10000;

        start = clock();

        mergesort(a, 0, n - 1);

        end = clock();

        time_taken =
            ((double)(end - start)) * 1000 / CLOCKS_PER_SEC;

        printf("%d\t%.2f\n", n, time_taken);
    }

    return 0;
}
```

Output :

```
Enter number of elements: 5
Enter elements:
25 80 3 5 90

Sorted elements:
3 5 25 80 90

N   Time(ms)
100 0.02
500 0.07
1000    0.16
2000    0.37
3000    0.50
```

Graph :

Leetcode 912 :



Code :

```
void merge(int nums[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)
        L[i] = nums[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = nums[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            nums[k++] = L[i++];
        else
            nums[k++] = R[j++];
    }

    while (i < n1)
        nums[k++] = L[i++];

    while (j < n2)
        nums[k++] = R[j++];
}

void mergeSort(int nums[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(nums, left, mid);
        mergeSort(nums, mid + 1, right);
```

```
        merge(nums, left, mid, right);
    }
}

int* sortArray(int* nums, int numsSize, int* returnSize) {
    mergeSort(nums, 0, numsSize - 1);

    *returnSize = numsSize;
    return nums;
}
```

Lab program 4: Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

C Program :

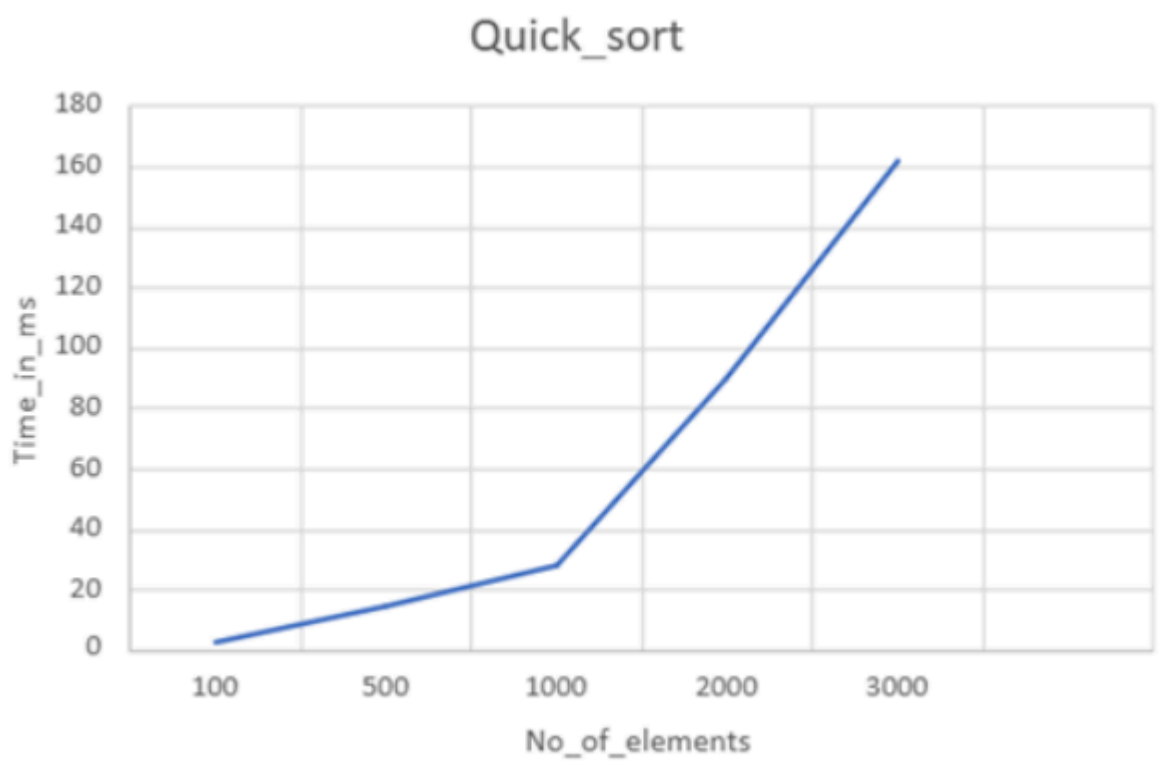
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int partition(int a[], int low, int high) {
   int pivot = a[low];
   int i = low + 1;
   int j = high;
   int temp;

   while (1) {
      while (i <= high && a[i] <= pivot)
         i++;

      while (a[j] > pivot)
         j--;

      if (i < j) {
         temp = a[i];
         a[i] = a[j];
         a[j] = temp;
      } else {
         break;
      }
   }

   temp = a[low];
   a[low] = a[j];
   a[j] = temp;

   return j;
}

void quicksort(int a[], int low, int high) {
   if (low < high) {
      int p = partition(a, low, high);

      quicksort(a, low, p - 1);
      quicksort(a, p + 1, high);
   }
}

int main() {
   int a[100000], b[100000];
   int n;
   int values[5] = {100, 500, 1000, 2000, 3000};

   clock_t start, end;
   double time_taken;
```

```
    srand(time(NULL));

    printf("Enter number of elements: ");
    scanf("%d", &n);

    if (n <= 0 || n > 100000) {
        printf("Invalid number of elements.\n");
        return 1;
    }
    printf("Enter elements:\n");

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quicksort(a, 0, n - 1);

    printf("\nSorted elements:\n");

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n\nN\tTime(ms)\n");

    for (int i = 0; i < 5; i++) {
        n = values[i];

        for (int j = 0; j < n; j++)
            a[j] = rand() % 10000;

        start = clock();

        for (int k = 0; k < 500; k++) {
            for (int j = 0; j < n; j++)
                b[j] = a[j];

            quicksort(b, 0, n - 1);
        }
        end = clock();
        time_taken = ((double)(end - start) * 1000.0) / CLOCKS_PER_SEC;

        printf("%d\t%.2f\n", n, time_taken);
    }
    return 0;
}
```

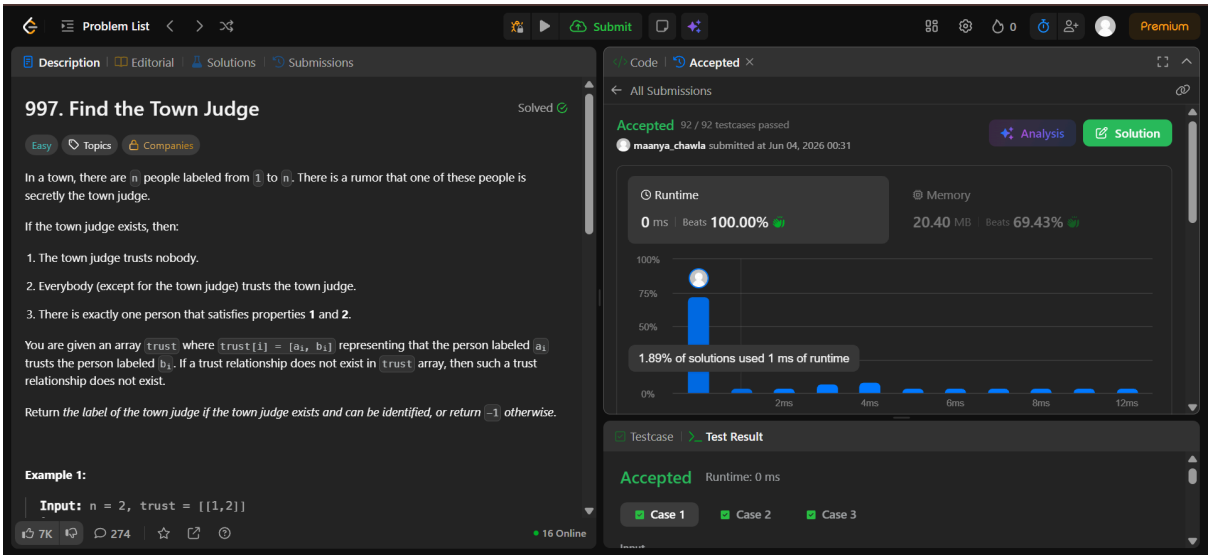
Output :

```
Enter number of elements: 10
Enter elements:
90 80 55 45 12 22 35 100 65 75

Sorted elements:
12 22 35 45 55 65 75 80 90 100

N   Time(ms)
100 1.05
500 6.50
1000    15.13
2000    32.53
3000    104.10
```

Graph :



Leetcode 997 :



Code :

```
int findJudge(int n, int** trust, int trustSize, int* trustColSize) {
    int indegree[n + 1];
    int outdegree[n + 1];

    for (int i = 0; i <= n; i++) {
        indegree[i] = 0;
        outdegree[i] = 0;
    }

    for (int i = 0; i < trustSize; i++) {
        int a = trust[i][0];
        int b = trust[i][1];

        outdegree[a]++;
        indegree[b]++;
    }

    for (int i = 1; i <= n; i++) {
        if (indegree[i] == n - 1 && outdegree[i] == 0)
            return i;
    }

    return -1;
}
```

Lab program 5: Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

C Program :

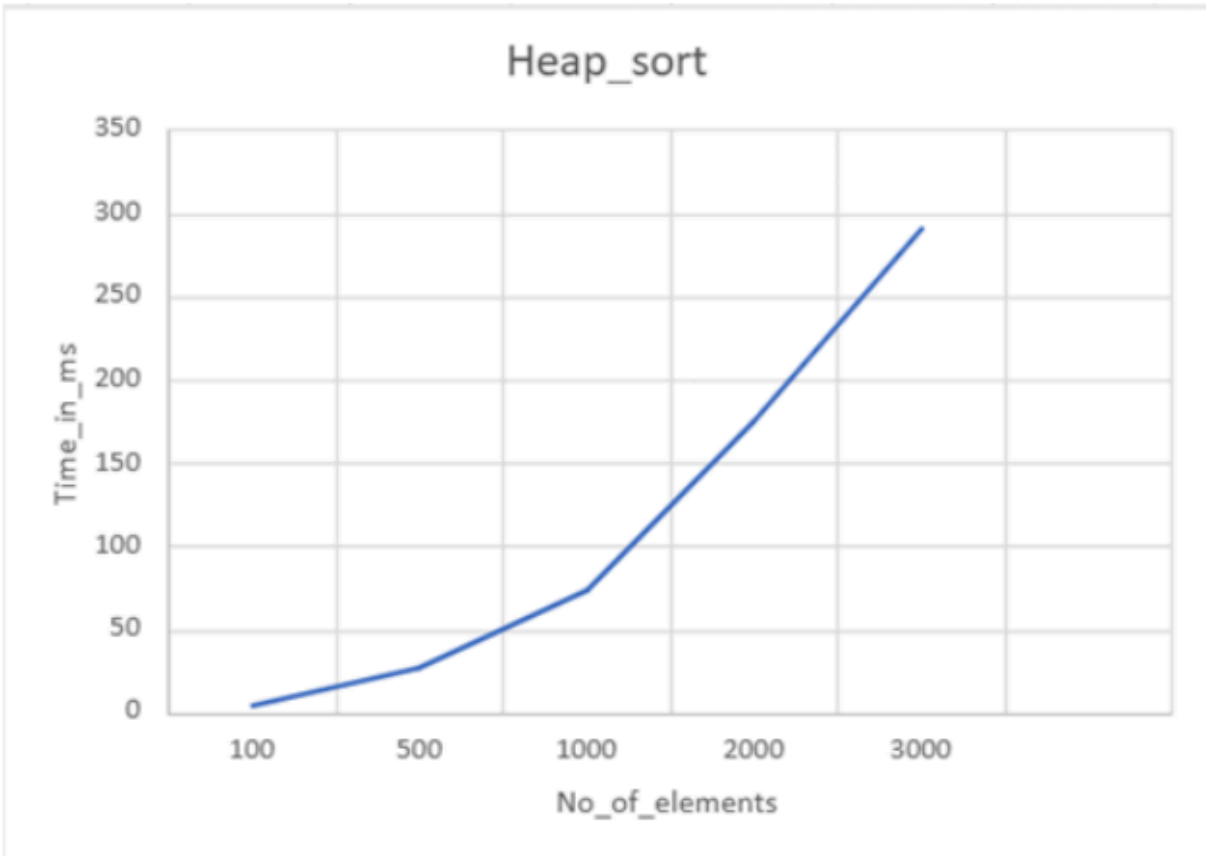
```
#include <stdio.h>
#include <time.h>

void heapify(int a[], int n, int i) {
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;

   if (left < n && a[left] > a[largest])
      largest = left;

   if (right < n && a[right] > a[largest])
      largest = right;

   if (largest != i) {
      int temp = a[i];
      a[i] = a[largest];
      a[largest] = temp;

      heapify(a, n, largest);
   }
}

void heapSort(int a[], int n) {


   for (int i = n/2 - 1; i >= 0; i--)
      heapify(a, n, i);


   for (int i = n-1; i > 0; i--) {

      int temp = a[0];
      a[0] = a[i];
      a[i] = temp;

      heapify(a, i, 0);
   }
}

int main() {
   int n;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   int a[n];

   printf("Enter elements:\n");
```

```
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    clock_t start, end;

    start = clock();

    heapSort(a, n);

    end = clock();

    printf("\nSorted Array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    double time_taken =
        ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\n\nTime Taken = %f seconds\n", time_taken);

    return 0;
}
```

Output :

```
Enter number of elements: 10
Enter elements:
100 25 65 15 80 75 65 35 25 89

Sorted elements:
15 25 25 35 65 65 75 80 89 100

N   Time(ms)
100 2.35
500 15.36
1000    39.20
2000    115.51
3000    211.95
```

Graph :



Lab program 6: Implement 0/1 Knapsack problem using dynamic programming.

C Program :

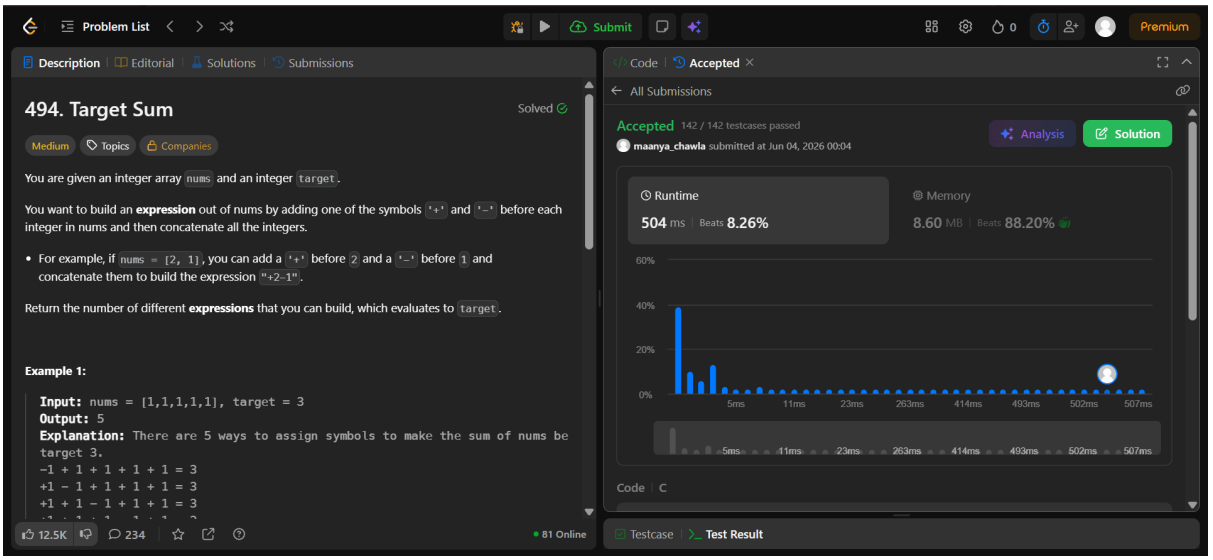
```
#include <stdio.h>

int max(int a, int b)
{
   return (a > b) ? a : b;
}

int knapsack(int W, int wt[], int val[], int n)
{
   int i, w;

   int K[n + 1][W + 1];

   for(i = 0; i <= n; i++)
   {
      for(w = 0; w <= W; w++)
      {
         if(i == 0 || w == 0)
         {
            K[i][w] = 0;
         }
         else if(wt[i - 1] <= w)
         {
            K[i][w] = max(
               val[i - 1] + K[i - 1][w - wt[i - 1]],
               K[i - 1][w]
            );
         }
         else
         {
            K[i][w] = K[i - 1][w];
         }
      }
   }

   return K[n][W];
}

int main()
{
   int n, W, i;

   printf("Enter number of items: ");
   scanf("%d", &n);

   int wt[n], val[n];

   printf("Enter weights of items: ");
   for(i = 0; i < n; i++)
      scanf("%d", &wt[i]);
```

```
    printf("Enter values of items: ");
    for(i = 0; i < n; i++)
        scanf("%d", &val[i]);

    printf("Enter knapsack capacity: ");
    scanf("%d", &W);

    printf("Maximum value = %d\n",
           knapsack(W, wt, val, n));

    return 0;
}
```

Output :

```
Enter number of items: 3
Enter weights of items: 10 20 30
Enter values of items: 80 120 160
Enter knapsack capacity: 50
Maximum value = 280
```

Leetcode 494 :



Code :

```
int dfs(int* nums, int numsSize, int index, int sum, int target) {
    if (index == numsSize) {
        return (sum == target) ? 1 : 0;
    }

    return dfs(nums, numsSize, index + 1, sum + nums[index], target) +
           dfs(nums, numsSize, index + 1, sum - nums[index], target);
}

int findTargetSumWays(int* nums, int numsSize, int target) {
    return dfs(nums, numsSize, 0, 0, target);
}
```

Lab program 7: Implement All Pair Shortest paths problem using Floyd's algorithm.

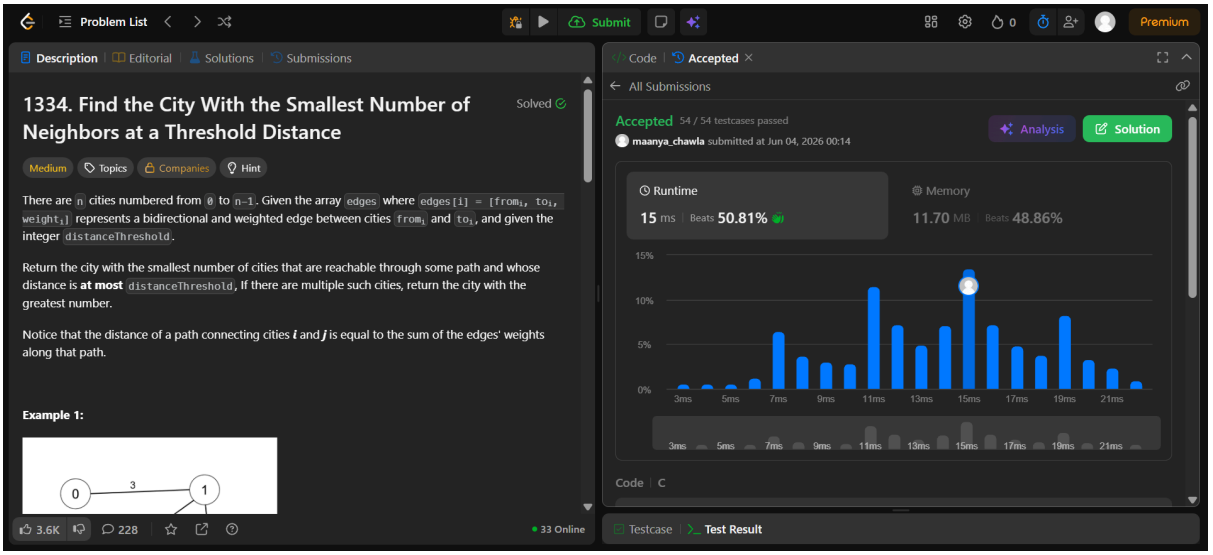
C Program :

```
#include <stdio.h>
#define MAX 100
#define INF 99999

void floyd(int graph[MAX][MAX], int n) {
   int dist[MAX][MAX];


   for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
         dist[i][j] = graph[i][j];


   for (int k = 0; k < n; k++) {
      for (int i = 0; i < n; i++) {
         for (int j = 0; j < n; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
               dist[i][j] = dist[i][k] + dist[k][j];
            }
         }
      }
   }


   printf("\nAll-Pairs Shortest Path Matrix:\n");
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         if (dist[i][j] == INF)
            printf("INF ");
         else
            printf("%3d ", dist[i][j]);
      }
      printf("\n");
   }
}

int main() {
   int n, graph[MAX][MAX];

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter adjacency matrix (use 99999 for INF):\n");
   for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
         scanf("%d", &graph[i][j]);

   floyd(graph, n);

   return 0;}
```

Output :

```
Enter number of vertices: 4
Enter adjacency matrix (use 99999 for INF):
0 5 99999 10
99999 0 3 99999
99999 99999 0 1
99999 99999 99999 0

All-Pairs Shortest Path Matrix:
  0   5   8   9
INF   0   3   4
INF INF   0   1
INF INF INF   0
```

Leetcode 1334 :



Code :

```
#define INF 1000000000
int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize,
                int distanceThreshold) {

    int dist[100][100];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j)
                dist[i][j] = 0;
            else
                dist[i][j] = INF;
        }
    }
```

```
    for (int i = 0; i < edgesSize; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        int w = edges[i][2];

        dist[u][v] = w;
        dist[v][u] = w;
    }

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] != INF &&
                    dist[k][j] != INF &&
                    dist[i][k] + dist[k][j] < dist[i][j]) {

                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    int answer = -1;
    int minReachable = n;

    for (int i = 0; i < n; i++) {
        int count = 0;

        for (int j = 0; j < n; j++) {
            if (i != j && dist[i][j] <= distanceThreshold)
                count++;
        }

        if (count <= minReachable) {
            minReachable = count;
            answer = i;
        }
    }

    return answer;
}
```

Lab program 8 (a) : Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

C Program :

```
#include <stdio.h>

#define MAX 10
#define INF 99999

int main() {
   int n;
   int graph[MAX][MAX];
   int selected[MAX] = {0};

   int edges = 0;
   int minCost = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter adjacency matrix (0 for no edge):\n");
   for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
         scanf("%d", &graph[i][j]);

   selected[0] = 1;

   printf("\nEdges in MST:\n");

   while (edges < n - 1) {

      int min = INF;
      int x = -1, y = -1;

      for (int i = 0; i < n; i++) {

         if (selected[i]) {

            for (int j = 0; j < n; j++) {

               if (!selected[j] &&
                  graph[i][j] &&
                  graph[i][j] < min) {

                  min = graph[i][j];
                  x = i;
                  y = j;
               }
            }
         }
      }

      printf("%d - %d : %d\n", x, y, graph[x][y]);
```

```
        minCost += graph[x][y];
        selected[y] = 1;
        edges++;
    }

    printf("\nMinimum Cost = %d\n", minCost);

    return 0;
}
```

Output :

```
Enter number of vertices: 4
Enter adjacency matrix (0 for no edge):
0 2 99999 6
2 0 3 8
99999 3 0 5
6 8 5 0

Edges in MST:
0 - 1 : 2
1 - 2 : 3
2 - 3 : 5

Minimum Cost = 10
```

Lab program 8 (b) : Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

C Program :

```
#include <stdio.h>
#define MAX 10

struct Edge {
   int u, v, w;
};

int parent[MAX];

// Find parent
int find(int i) {
   while (parent[i] != i)
      i = parent[i];
   return i;
}


void unite(int a, int b) {
   parent[a] = b;
}


void sort(struct Edge e[], int n) {
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (e[j].w > e[j + 1].w) {
            struct Edge temp = e[j];
            e[j] = e[j + 1];
            e[j + 1] = temp;
         }
      }
   }
}

int main() {
   int n;
   int graph[MAX][MAX];
   struct Edge edges[MAX * MAX];

   int edgeCount = 0;
   int minCost = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter adjacency matrix (0 for no edge):\n");

   for (int i = 0; i < n; i++) {
      parent[i] = i;
```

```
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);

            if (graph[i][j] != 0 && i < j) {
                edges[edgeCount].u = i;
                edges[edgeCount].v = j;
                edges[edgeCount].w = graph[i][j];
                edgeCount++;
            }
        }
    }

    sort(edges, edgeCount);

    printf("\nEdges in MST:\n");

    int count = 0;

    for (int i = 0; i < edgeCount && count < n - 1; i++) {

        int u = find(edges[i].u);
        int v = find(edges[i].v);

        if (u != v) {
            printf("%d - %d : %d\n",
                edges[i].u,
                edges[i].v,
                edges[i].w);

            minCost += edges[i].w;
            unite(u, v);
            count++;
        }
    }

    printf("\nMinimum Cost = %d\n", minCost);

    return 0;
}
```

Output :

```
Enter number of vertices: 4
Enter adjacency matrix (0 for no edge):
99999 2 99999 6
2 99999 3 8
99999 3 99999 5
6 8 5 99999

Edges in MST:
0 - 1 : 2
1 - 2 : 3
2 - 3 : 5

Minimum Cost = 10
```

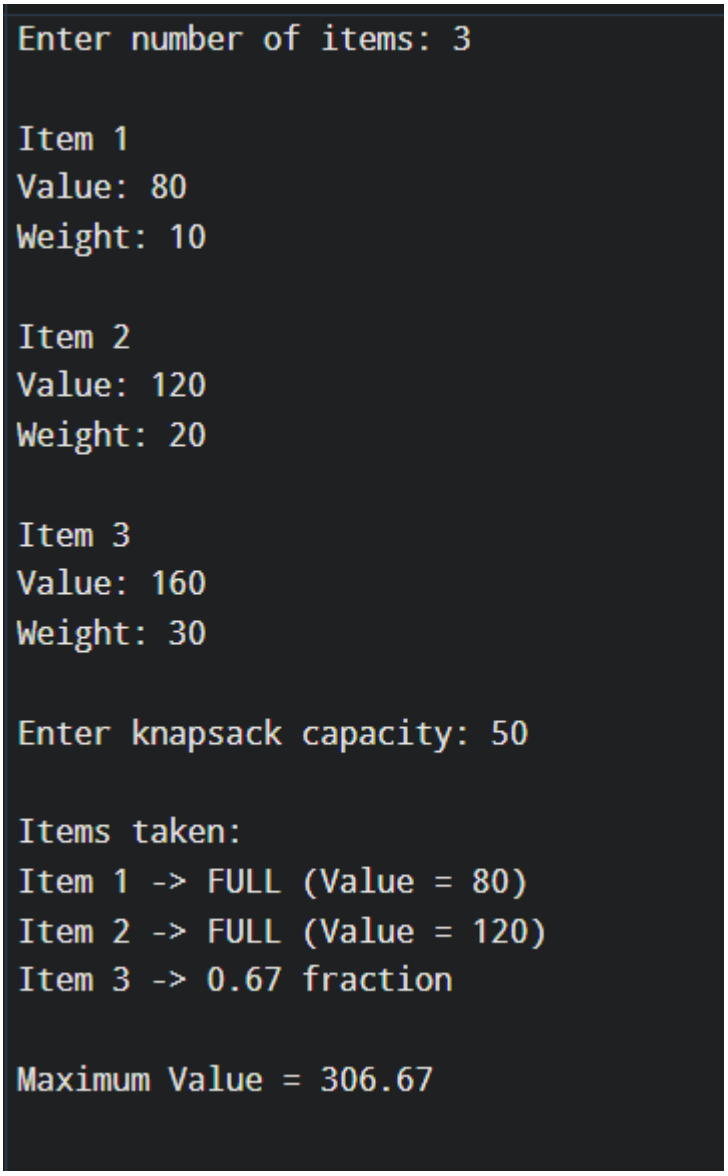
Lab program 9 : Implement Fractional Knapsack using Greedy technique.

C Program :

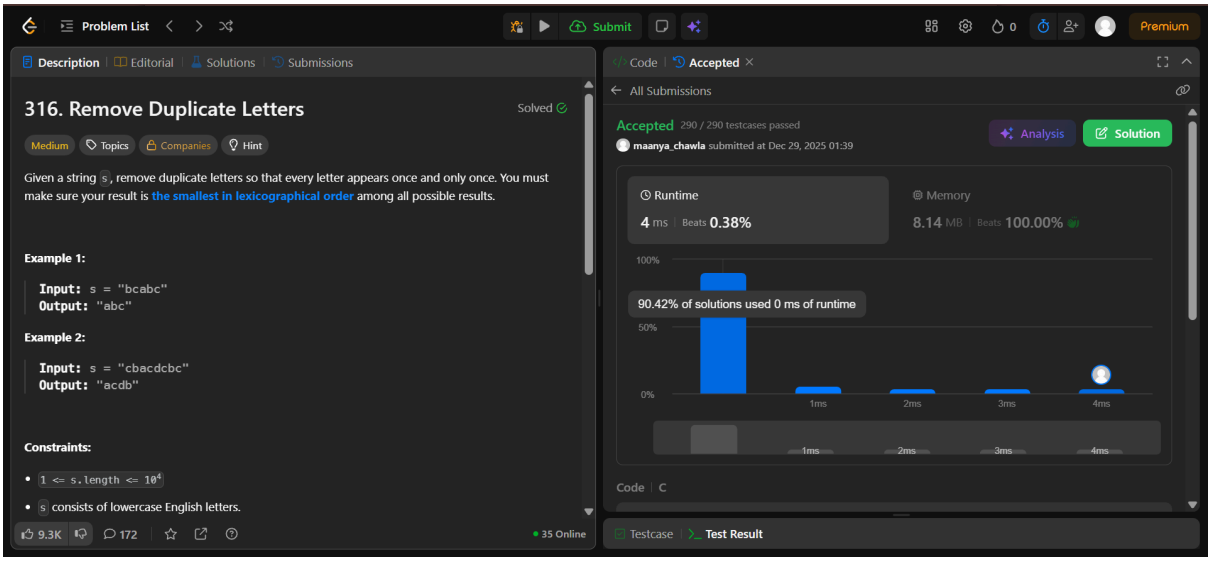
```
#include <stdio.h>

struct Item {
   int weight, value;
   float ratio;
};

void sort(struct Item arr[], int n) {
   for (int i = 0; i < n-1; i++)
      for (int j = 0; j < n-i-1; j++)
         if (arr[j].ratio < arr[j+1].ratio) {
            struct Item temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
         }
}

int main() {
   int n, capacity;
   struct Item arr[100];

   printf("Enter number of items: ");
   scanf("%d", &n);

   for (int i = 0; i < n; i++) {
      printf("\nItem %d\n", i+1);
      printf("Value: ");
      scanf("%d", &arr[i].value);
      printf("Weight: ");
      scanf("%d", &arr[i].weight);
      arr[i].ratio = (float)arr[i].value / arr[i].weight;
   }

   printf("\nEnter knapsack capacity: ");
   scanf("%d", &capacity);

   sort(arr, n);

   float totalValue = 0.0;

   printf("\nItems taken:\n");

   for (int i = 0; i < n; i++) {
      if (capacity == 0) break;

      if (arr[i].weight <= capacity) {

         capacity -= arr[i].weight;
         totalValue += arr[i].value;
         printf("Item %d -> FULL (Value = %d)\n", i+1, arr[i].value);
```

```
        } else {

            float fraction = (float)capacity / arr[i].weight;
            totalValue += arr[i].value * fraction;
            printf("Item %d -> %.2f fraction\n", i+1, fraction);
            capacity = 0;
        }
    }

    printf("\nMaximum Value = %.2f\n", totalValue);

    return 0;
}
```

Output:



Leetcode 316 :



Code :

```
char* removeDuplicateLetters(char* s) {
int count[26] = {0};
char stack[27];
int top = -1;
// count frequency
for (int i = 0; s[i]; i++) {
count[s[i] - 'a']++;
}
for (int i = 0; s[i]; i++) {
char ch = s[i];
int c = ch - 'a';
count[c]--;
// check if ch already in stack (bounded linear scan)
int alreadyIn = 0;
for (int j = 0; j <= top; j++) {
if (stack[j] == ch) {
alreadyIn = 1;
break;
}
}
if (alreadyIn)
continue;
// pop while valid
while (top >= 0 &&
stack[top] > ch &&
count[stack[top] - 'a'] > 0) {
top--;
}
stack[++top] = ch;
}
stack[top + 1] = '\0';
return strdup(stack);
}
```

Lab program 10 : From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

C Program :

```
#include <stdio.h>
#define MAX 100
#define INF 99999

int minDist(int dist[], int visited[], int n) {
   int min = INF, index = -1;

   for (int i = 0; i < n; i++) {
      if (!visited[i] && dist[i] < min) {
         min = dist[i];
         index = i;
      }
   }
   return index;
}

void dijkstra(int graph[MAX][MAX], int n, int src) {
   int dist[MAX], visited[MAX];


   for (int i = 0; i < n; i++) {
      dist[i] = INF;
      visited[i] = 0;
   }
   dist[src] = 0;

   for (int i = 0; i < n-1; i++) {
      int u = minDist(dist, visited, n);
      visited[u] = 1;

      for (int v = 0; v < n; v++) {
         if (!visited[v] && graph[u][v] &&
            dist[u] + graph[u][v] < dist[v]) {

            dist[v] = dist[u] + graph[u][v];
         }
      }
   }


   printf("\nVertex\tDistance from Source\n");
   for (int i = 0; i < n; i++)
      printf("%d\t%d\n", i, dist[i]);
}

int main() {
   int n, src;
   int graph[MAX][MAX];
```

```
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (0 for no edge):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    printf("Enter source vertex: ");
    scanf("%d", &src);

    dijkstra(graph, n, src);

    return 0;
}
```

Output :

```
Enter number of vertices: 4
Enter adjacency matrix (0 for no edge):
0 1 4 9999
1 0 2 6
4 2 0 3
9999 6 3 0
Enter source vertex: 0

Vertex  Distance from Source
0   0
1   1
2   3
3   6
```

Lab program 11 : Implement “N-Queens Problem” using Backtracking.

C Program :

```
#include <stdio.h>
#include <stdlib.h>

int x[20], n;

int place(int k, int i) {
   for (int j = 1; j < k; j++)
      if (x[j] == i || abs(x[j] - i) == abs(j - k))
         return 0;
   return 1;
}

void queen(int k) {
   for (int i = 1; i <= n; i++) {
      if (place(k, i)) {
         x[k] = i;
         if (k == n) {
            for (int j = 1; j <= n; j++)
               printf("%d ", x[j]);
            printf("\n");
         } else
            queen(k + 1);
      }
   }
}

int main() {
   printf("Enter n: ");
   scanf("%d", &n);
   queen(1);
   return 0;
}
```

Output :

```
Enter n: 4
2 4 1 3
3 1 4 2
```